D6 · Object evaluation engine

The second engine of the platform. The scan engine (D3) finds objects from open data; this engine values one mandated object from its real documents — down to an offer price.

Where it sits in the system

scan engine finds → client mandates → documents arrive → **evaluation engine values** → offer is written. It runs per object, only after the mandate, when Grundbuch, plans, Mietverträge and photos exist. Its results feed C5 (client sees value + report), A2 (agent works with it), and the bank package (A4). It writes to the same database as everything else — no private copies, every run logged in activity_log.

The pipeline — ten layers, one direction

Each layer is a function: it reads what the previous layers wrote, adds its result to the object's evaluation record, and never reaches back. If a layer's inputs are missing, it records what is missing and lowers the confidence score instead of guessing.

#	Layer	In → Out	MVP status
1	Object intake	Address, Flurstück, Grundbuchauszug, GVZ, photos, condition, Mietverträge, plans, energy certificate, areas, rent, vacancy → one digital object file	Build — upload forms + fields on the existing documents table
2	Document reading	Grundbuch (owner, rights, Grundschulden, restrictions, easements), plans (areas, units, structure, use), Mietverträge (rent, duration, tenant risk, indexation) → structured fact sheet	Build assisted — system extracts and suggests, a human confirms every fact before it counts (see rule box)
3	Data verification	Cross-checks: plan area vs exposé, Grundbuch vs seller info, rent vs contracts, GVZ plausibility, photo coverage → data confidence % + missing-documents list	Build — deterministic checklist, no AI needed
4	Location & land value	Bodenrichtwert, micro/macro location, transport, rent level, Bebauungsplan, ALKIS → land value € + location score	Mostly exists — reuse D2 layers + scan-engine data for this object; Bodenrichtwert manual until BORIS licensing is cleared
5	Zustand / capex	13 components (roof, facade, windows, heating, electricity, pipes, bathrooms, floors, basement, moisture, fire protection, energy upgrade, common areas), each: condition → urgency → cost → buffer → capex range	Build — cost table per component per m²/unit, editable by the office. The most important layer: wrong capex = wrong everything
6	Income	Current rent, market rent, vacancy, non-recoverables, maintenance → current NOI + potential NOI + rent upside	Build — pure arithmetic on confirmed facts
7	Valuation (3 German methods)	Ertragswert (income capitalization), Sachwert (land + replacement – depreciation), Vergleichswert (adjusted comparables) → three values + recommended range	Build Ertragswert + Sachwert fully; Vergleichswert semi-manual (office enters comparables, system adjusts)
8	Bank logic	Sustainable rent, conservative assumptions, risk buffers → bank-style conservative value + realistic loan basis (60–70%)	Build — a haircut ruleset on layer 7, clearly labelled 'not an official bank valuation'

#	Layer	In → Out	MVP status
9	Investor scenarios	Buy as-is / renovate-hold / renovate-sell / energy upgrade / rent increase / densification → profit per scenario + risk-adjusted max offer	Build 3 scenarios (as-is, renovate-hold, renovate-sell); the rest after MVP
10	Report & decision	Everything above → one PDF: summary, checked + missing documents, risks, capex range, three values, bank value, offer recommendation, confidence % → Buy / Negotiate / Reject / Need documents / Need inspection	Build — reuse the mandate PDF generator; this report is the product the client actually holds

The MVP cut — six modules, in this build order

Order	Module	Covers layers	Why this order
1	Object File	1	Everything else reads from it; it is mostly forms on existing tables
2	Fact Sheet + Verification	2 + 3	Extraction and checking are one workflow for the human confirming facts
3	Capex Engine	5	Highest impact on value; needs only the component cost table and condition inputs
4	Income Engine	6	Small, pure arithmetic — an afternoon once facts are confirmed
5	Valuation Engine	4 + 7 + 8	Land value, three methods and the bank haircut belong together
6	Report Engine	9 + 10	Scenarios + report last, because they consume everything else

Data model additions (extends D2)

Table	One row per	Key columns
`evaluations`	evaluation run of one object	eval_id, object_id, mandate_id, status, confidence_pct, missing_docs[], created_at
`facts`	confirmed fact	eval_id, key (e.g. living_area_m2, annual_net_rent), value, source_doc_id, extracted_by (system/human), confirmed_by, confirmed_at
`capex_items`	component estimate	eval_id, component, condition, urgency, cost_low, cost_high, note
`valuations`	method result	eval_id, method (ertragswert/sachwert/vergleichswert/bank), value_low, value_high, inputs_json
`scenarios`	strategy scenario	eval_id, scenario, total_cost, exit_value, profit, max_offer
`cost_table`	component unit cost (config)	component, unit (m2/unit/flat), cost_low, cost_high, updated_by — office-editable, versioned

Five rules that prevent the classic failures

■ Human-confirmed facts only: layer 2 extraction (OCR + model) always proposes, never decides. A fact without confirmed_by does not enter any calculation. This keeps the system honest while extraction quality grows.
■ No guessing on gaps: missing input → the layer writes it to missing_docs and lowers confidence_pct. The report says 'Need more documents' instead of inventing a number.
■ Every number explains itself: valuations and scenarios store inputs_json, so for any value we can print the formula, the inputs and the fact sources behind it — the same rule as the scan engine.
■ Ranges, not points: capex, values and offers are always low–high ranges. A single number pretends a precision we do not have; a range is defensible in front of a bank.
■ One cost table, owned by the office: capex prices live in cost_table, never in code. When Stuttgart prices move, the office edits a row and the next evaluation is current.

Definition of done

■ One test object (use the 28-apartment mixed-use case) goes through all six modules end-to-end.

■ The fact sheet shows every fact with its source document and who confirmed it.

■ Changing one capex component or the cost table re-computes value, bank value and offer instantly.

■ The final PDF report contains: values (three methods), bank value, capex range, risks, missing documents, confidence %, and one recommendation with a safe offer range and a do-not-exceed price.

■ A second run on the same inputs produces the same report (deterministic, like D3).